Agents (智能体)

智能体将语言模型与 工具 相结合，创建出能够推理任务、决定使用哪些工具并迭代地朝着解决方案推进的系统。

`create_agent` 提供了一个可用于生产环境的智能体实现。

LLM 智能体在循环中运行工具以实现目标。智能体持续运行直到满足停止条件——即当模型输出最终结果或达到迭代限制时。

核心组件

模型

模型 是智能体的推理引擎。它可以通过多种方式指定，支持静态和动态模型选择。

静态模型

静态模型在创建智能体时配置一次，并在整个执行过程中保持不变。这是最常见且直接的方法。

使用模型标识符字符串（格式为 `provider:model`，如 `openai:gpt-5`）初始化静态模型：

```
from langchain.agents import create_agent

agent = create_agent("openai:gpt-5", tools=tools)
```

> **提示**: 模型标识符字符串支持自动推断（例如 `"gpt-5"` 将被推断为 `"openai:gpt-5"`）。参考文档查看完整的模型标识符字符串映射列表。

为了更好地控制模型配置，可以使用提供者包直接初始化模型实例。以下示例使用 `ChatOpenAI`。其他可用的聊天模型类参见聊天模型文档。

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5",
    temperature=0.1,
    max_tokens=1000,
    timeout=30
    # ... (其他参数)
)
agent = create_agent(model, tools=tools)
```

模型实例让您完全控制配置。当需要设置特定的参数（如 `temperature`、`max_tokens`、`timeouts`、`base_url` 等提供者特定设置）时使用它们。

动态模型

动态模型根据当前的状态和上下文在运行时选择。这支持复杂的路由逻辑和成本优化。

要使用动态模型，创建使用 `@wrap_model_call` 装饰器的中间件来修改请求中的模型：

```
from langchain_openai import ChatOpenAI
```

```python
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse


basic_model = ChatOpenAI(model="gpt-4.1-mini")
advanced_model = ChatOpenAI(model="gpt-4.1")

@wrap_model_call
def dynamic_model_selection(request: ModelRequest, handler) -> ModelResponse:
    """根据对话复杂度选择模型。"""
    message_count = len(request.state["messages"])

    if message_count > 10:
        # 对较长对话使用高级模型
        model = advanced_model
    else:
        model = basic_model

    return handler(request.override(model=model))

agent = create_agent(
    model=basic_model,  # 默认模型
    tools=tools,
    middleware=[dynamic_model_selection]
)
```

> **警告**: 使用结构化输出时不支持预绑定模型（已调用 `bind_tools` 的模型）。如果需要使用结构化输出进行动态模型选择，请确保传递给中间件的模型未预绑定。

工具

工具赋予智能体采取行动的能力。智能体超越了简单的仅模型工具绑定，支持：

- 顺序执行多个工具调用（由单个提示触发）
- 适当时并行工具调用
- 基于先前结果的动态工具选择
- 工具重试逻辑和错误处理
- 工具调用之间的状态持久性

静态工具

静态工具在创建智能体时定义，并在整个执行过程中保持不变。

```python
from langchain.tools import tool
from langchain.agents import create_agent


@tool
def search(query: str) -> str:
    """搜索信息。"""
    return f"结果: {query}"

```

```
@tool
def get_weather(location: str) -> str:
    """获取某个地点的天气信息。"""
    return f"{location} 天气：晴，72°F"

agent = create_agent(model, tools=[search, get_weather])
```

如果提供空工具列表，智能体将仅包含单个 LLM 节点，没有工具调用能力。

动态工具

使用动态工具时，智能体可用的工具集在运行时修改，而不是一开始就全部定义。

方法一：过滤预注册的工具

当所有可能的工具在创建智能体时已知，可以预注册它们，并根据状态、权限或上下文动态过滤暴露给模型的工具。

按状态过滤:

```
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@wrap_model_call
def state_based_tools(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
    """根据对话状态过滤工具。"""
    state = request.state
    is_authenticated = state.get("authenticated", False)
    message_count = len(state["messages"])

    if not is_authenticated:
        tools = [t for t in request.tools if t.name.startswith("public_")]
        request = request.override(tools=tools)
    elif message_count < 5:
        tools = [t for t in request.tools if t.name != "advanced_search"]
        request = request.override(tools=tools)

    return handler(request)

agent = create_agent(
    model="gpt-4.1",
    tools=[public_search, private_search, advanced_search],
    middleware=[state_based_tools]
)
```

按存储过滤:

```
from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
```

```python
from typing import Callable
from langgraph.store.memory import InMemoryStore

@dataclass
class Context:
    user_id: str

@wrap_model_call
def store_based_tools(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
    """根据存储偏好过滤工具。"""
    user_id = request.runtime.context.user_id

    store = request.runtime.store
    feature_flags = store.get(("features",), user_id)

    if feature_flags:
        enabled_features = feature_flags.value.get("enabled_tools", [])
        tools = [t for t in request.tools if t.name in enabled_features]
        request = request.override(tools=tools)

    return handler(request)

agent = create_agent(
    model="gpt-4.1",
    tools=[search_tool, analysis_tool, export_tool],
    middleware=[store_based_tools],
    context_schema=Context,
    store=InMemoryStore()
)
```

按运行时上下文过滤:

```python
from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@dataclass
class Context:
    user_role: str

@wrap_model_call
def context_based_tools(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
    """根据运行时上下文权限过滤工具。"""
    if request.runtime is None or request.runtime.context is None:
        user_role = "viewer"
    else:
        user_role = request.runtime.context.user_role
```

```python

    if user_role == "admin":
        pass
    elif user_role == "editor":
        tools = [t for t in request.tools if t.name != "delete_data"]
        request = request.override(tools=tools)
    else:
        tools = [t for t in request.tools if t.name.startswith("read_")]
        request = request.override(tools=tools)

    return handler(request)

agent = create_agent(
    model="gpt-4.1",
    tools=[read_data, write_data, delete_data],
    middleware=[context_based_tools],
    context_schema=Context
)
```

方法二：运行时工具注册

当工具在运行时发现或创建时，需要同时注册工具并动态处理其执行。

```python
from langchain.tools import tool
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware, ModelRequest, ToolCallRequest

@tool
def calculate_tip(bill_amount: float, tip_percentage: float = 20.0) -> str:
    """计算账单的小费金额。"""
    tip = bill_amount * (tip_percentage / 100)
    return f"小费：${tip:.2f}，总计：${bill_amount + tip:.2f}"

class DynamicToolMiddleware(AgentMiddleware):
    """注册和处理动态工具的中间件。"""

    def wrap_model_call(self, request: ModelRequest, handler):
        updated = request.override(tools=[*request.tools, calculate_tip])
        return handler(updated)

    def wrap_tool_call(self, request: ToolCallRequest, handler):
        if request.tool_call["name"] == "calculate_tip":
            return handler(request.override(tool=calculate_tip))
        return handler(request)

agent = create_agent(
    model="gpt-4o",
    tools=[get_weather],
    middleware=[DynamicToolMiddleware()],
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "计算 85 美元的 20% 小费"}]
})
```

工具错误处理

```python
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage


@wrap_tool_call
def handle_tool_errors(request, handler):
    """使用自定义消息处理工具执行错误。"""
    try:
        return handler(request)
    except Exception as e:
        return ToolMessage(
            content=f"工具错误：请检查输入并重试。({str(e)})",
            tool_call_id=request.tool_call["id"]
        )

agent = create_agent(
    model="gpt-4.1",
    tools=[search, get_weather],
    middleware=[handle_tool_errors]
)
```

ReAct 循环中的工具使用

智能体遵循 ReAct（"推理 + 行动"）模式，在简短的推理步骤与有针对性的工具调用之间交替，并将结果观察反馈到后续决策中，直到能够提供最终答案。

系统提示

您可以通过提供提示来塑造智能体处理任务的方式。

```python
agent = create_agent(
    model,
    tools,
    system_prompt="你是一个乐于助人的助手。保持简洁准确。"
)
```

动态系统提示

```python
from typing import TypedDict

from langchain.agents import create_agent
from langchain.agents.middleware import dynamic_prompt, ModelRequest


class Context(TypedDict):
    user_role: str

@dynamic_prompt
def user_role_prompt(request: ModelRequest) -> str:
    """根据用户角色生成系统提示。"""
    user_role = request.runtime.context.get("user_role", "user")
```

```
    base_prompt = "你是一个乐于助人的助手。"

    if user_role == "expert":
        return f"{base_prompt} 提供详细的技术响应。"
    elif user_role == "beginner":
        return f"{base_prompt} 简单解释概念，避免行话。"

    return base_prompt

agent = create_agent(
    model="gpt-4.1",
    tools=[web_search],
    middleware=[user_role_prompt],
    context_schema=Context
)
```

名称

为智能体设置可选的名称。在多智能体系统中将智能体添加为子图时，这用作节点标识符：

```
agent = create_agent(
    model,
    tools,
    name="research_assistant"
)
```

> **警告**: 智能体名称最好使用 `snake_case` （如 `research_assistant` 而非 `Research Assistant`）。

调用

可以通过将更新传递给其状态来调用智能体：

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "旧金山的天气怎么样？"}]}
)
```

高级概念

结构化输出

ToolStrategy

`ToolStrategy` 使用人工工具调用生成结构化输出。这适用于任何支持工具调用的模型。

```
from pydantic import BaseModel
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy


class ContactInfo(BaseModel):
    name: str
    email: str
```

```
    phone: str

agent = create_agent(
    model="gpt-4.1-mini",
    tools=[search_tool],
    response_format=ToolStrategy(ContactInfo)
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "从以下信息提取联系方式: John Doe, john@example.com, (555) 123-4567"}]
})

result["structured_response"]
```

ProviderStrategy

`ProviderStrategy` 使用模型提供者的原生结构化输出生成。

```
from langchain.agents.structured_output import ProviderStrategy

agent = create_agent(
    model="gpt-4.1",
    response_format=ProviderStrategy(ContactInfo)
)
```

记忆

智能体通过消息状态自动维护对话历史。您还可以配置智能体使用自定义状态模式在对话期间记住额外信息。

通过中间件定义状态

```
from langchain.agents import AgentState
from langchain.agents.middleware import AgentMiddleware
from typing import Any


class CustomState(AgentState):
    user_preferences: dict

class CustomMiddleware(AgentMiddleware):
    state_schema = CustomState
    tools = [tool1, tool2]

    def before_model(self, state: CustomState, runtime) -> dict[str, Any] | None:
        ...

agent = create_agent(
    model,
    tools=tools,
    middleware=[CustomMiddleware()]
)
```

通过 `state_schema` 定义状态

```python
from langchain.agents import AgentState


class CustomState(AgentState):
    user_preferences: dict

agent = create_agent(
    model,
    tools=[tool1, tool2],
    state_schema=CustomState
)
```

> **注意**: 从 `langchain 1.0` 开始，自定义状态模式**必须**是 `TypedDict` 类型。

流式传输

```python
from langchain.messages import AIMessage, HumanMessage

for chunk in agent.stream({
    "messages": [{"role": "user", "content": "搜索 AI 新闻并总结发现"}]
}, stream_mode="values"):
    latest_message = chunk["messages"][-1]
    if latest_message.content:
        if isinstance(latest_message, HumanMessage):
            print(f"用户: {latest_message.content}")
        elif isinstance(latest_message, AIMessage):
            print(f"智能体: {latest_message.content}")
    elif latest_message.tool_calls:
        print(f"调用工具: {[tc['name'] for tc in latest_message.tool_calls]}")
```

中间件

中间件为在执行的各个阶段自定义智能体行为提供了强大的扩展性。您可以使用中间件来：

- 在调用模型之前处理状态（例如消息修剪、上下文注入）
- 修改或验证模型的响应（例如防护栏、内容过滤）
- 使用自定义逻辑处理工具执行错误
- 根据状态或上下文实现动态模型选择
- 添加自定义日志记录、监控或分析